

Dialog Act Classification for a Task-Oriented Restaurant Recommendation System

Anouk van Ladesteijn
Huub Statius Muller
Yanpeng Wang
Ruben Wijmenga

A.R.VANLADESTEIJN@UU.NL
H.B.STATIUSMULLER@STUDENTS.UU.NL
Y.WANG36@STUDENTS.UU.NL
R.WIJMENG@STUDENTS.UU.NL

Abstract

In this paper, we evaluate several dialog-act classifiers for a task-oriented restaurant recommendation system and compare their results. We compare a baseline rule-based classifier, Logistic Regression and Multi-Layer Perceptron classifiers using Bag-of-Words representations, and variants built on top of frozen word embeddings. Duplicate-aware grouped evaluation shows that the frozen-embedding MLP gives the best results.

1. Introduction

Task-oriented dialog systems rely on accurately recognizing the intention of the user to steer downstream conversational logic. The user’s utterances are mapped to dialog acts. Dialog acts are abstract representations of communicative functions such as confirming, greeting or requesting information and so on. Because the dialog act classifier operates at the entry point of the pipeline, an incorrect classification leads to complete failures in context tracking and dialog state management. Selecting a robust, accurate classification approach is therefore critical for the overall dialog system.

In this paper, we evaluate and compare multiple classification approaches using a dataset derived from the Dialog State Tracking Challenge 2 (DSTC2) (Henderson et al., 2014), comprising 15 distinct dialog act categories. We establish a rule-based baseline classifier using keyword matching. We then implement two machine learning classifiers using Logistic Regression (LR) and a Multi-Layer Perceptron (MLP). We evaluated these across two input representations: traditional Bag-of-Words (BoW) and dense frozen embeddings extracted from a pretrained DistilBERT transformer (Sanh et al., 2019). Additionally, to address the challenge of identical utterances causing data leakage, we compare a standard stratified split against a grouped stratified split.

The paper is structured as follows. Section 2 outlines the overall system architecture and key design decisions. Section 3 describes the experimental setup, feature extraction methods, and quantitative evaluation results. Section ?? presents a detailed error analysis, exploring data leakage, difficult cases, and error propagation. Finally, Section ?? summarizes our core contributions and findings to inform the implementation of the downstream dialog framework.

2. System architecture

All models use the same inputs and outputs, receiving an utterance converted to lowercase and stripped of punctuation, and outputting one of the 15 dialog-act labels (appendix A). We tested the models on the original stratified split and on a grouped split. In the grouped split, duplicate utterances are kept together, so the same sentence cannot occur in both training and test data. The 15-class dataset is strongly imbalanced: **inform**, **request**, and **thankyou** dominate, while several acts have only a few examples. The original split contains 20,398 training and 3,600 test rows. The grouped split produces 20,250 training and 3,748 test rows. Because **reqmore** is extremely rare, the grouped test set contains no example of it; this is an explicit limitation of the evaluation.

The rule-based model looks for certain predefined keywords in the utterance, which were manually selected by looking at common patterns in the data. The acts with the most specific keywords are checked first, to minimize the amount of false positives ('what about' in **reqalts** is more specific than 'what' in **request**). Since **inform** acts contain the most varied utterances, it is used as the default case.

For the classifiers using Bag-of-Words, **CountVectorizer** is initialized on the training vocabulary and a special Out of Vocabulary (OOV) token, with a list of predefined stop words that are to be ignored by the vectorizer. A Logistic Regression classifier and a one-layer MLP classifier are trained using these BoW representations of the training utterances. When classifying new (test) utterances, any words not in the training vocabulary are replaced by an OOV token.

Different versions of the two classifiers are trained on frozen word embeddings from **distilbert-base-uncased**. The frozen word embeddings are generated in batches where duplicates are removed and the models share the encoder and cached embeddings to improve performance.

3. Experiments and results

3.1. Model and split comparison

Table 1 shows that the MLP classifier with frozen embeddings performs the best across the board, both when trained on the original stratified split and on the grouped stratified split. The lower scores on the grouped split show that repeated utterances in the original split make classification easier. The grouped split therefore gives a more realistic estimate of performance on unseen utterances.

3.2. Error analysis

For the grouped frozen-MLP model, frequent classes such as **inform**, **request**, and **reqalts** have high F1 scores. Rare classes are more difficult. For example, **bye** has zero recall on 17 examples. The real test data also contains utterances that are difficult to classify. For example, "okay thank you and good bye" is labelled **bye**, but is predicted as **thankyou**. The utterance contains both a goodbye and a thank-you, which makes the single gold label difficult to predict. Another example is "correct", which is labelled **affirm** but can be difficult because it is very short and depends strongly on the previous system turn. The utterance "barbecue" is labelled **inform**, but also contains very little information by itself.

System and split	Accuracy
Rule baseline, Orig.	0.8400
BoW LR, Orig	0.97525
BoW LR, Group	0.9400
BoW MLP, Orig.	0.9825
BoW MLP, Group	0.9650
Frozen LR, Orig.	0.9864
Frozen LR, Group.	0.9560
Frozen MLP, Orig.	0.9878
Frozen MLP, Group	0.9813

Table 1: Evaluation results

4. Discussion

There are several limitations to the evaluation. The dataset is strongly imbalanced, so some dialog acts occur much more often than others. This means that high overall accuracy does not always mean that all classes are classified well. The grouped split also contains no examples of `reqmore`, so the performance of this class cannot be evaluated there. The dataset also assigns only one dialog-act label to each utterance. However some utterances could belong to more than one label, which can make the model prediction less clear.

5. Conclusion

The rule-based classifier, Logistic Regression, and an MLP were compared using both Bag-of-Words features and frozen DistilBERT embeddings. The models were evaluated on the original split and on a grouped split where duplicate utterances could not appear in both training and test data.

The frozen-embedding MLP performed best overall. On the grouped split, it achieved an accuracy of 0.9813. The grouped results were generally lower than the original-split results, showing that duplicate utterances can make the original evaluation easier.

References

- M. Henderson, B. Thomson, and J. D. Williams. The second dialog state tracking challenge. In *Proceedings of SIGDIAL*, 2014.
- V. Sanh, L. Debut, J. Chaumond, and T. Wolf. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*, 2019.

Appendix A. Data

The source file `dialog_acts.dat` stores one label followed by one utterance per line. Text is lowercase and stripped of punctuation. The dataset contains 23,998 rows, 4,823 unique utterances, and 15 labels: `ack`, `affirm`, `bye`, `confirm`, `deny`, `hello`, `inform`, `negate`, `null`, `repeat`, `reqalts`, `reqmore`, `request`, `restart`, and `thankyou`. The grouped stratified split greedily assigns complete duplicate groups, in decreasing order of group size, while approximating the per-class 85/15 target.

Appendix B. System details

The rule baseline uses manually selected keyword lists and a specificity order. BoW LR uses scikit-learn Logistic Regression; BoW MLP uses one hidden layer with 100 units and at most 300 iterations. The frozen encoder is `distilbert-base-uncased`, with maximum sequence length 64, masked mean pooling, batch size 64, GPU support, and a cache keyed by unique utterance. Frozen LR uses at most 1,000 iterations.

Appendix C. Difficult test cases

C.1. Difficult test set 1

1. “yeah, that sounds right” — `affirm`
2. “nope, not that one” — `negate`
3. “what’s the address again?” — `request`
4. “any other options?” — `reqalts`
5. “actually, let’s start over” — `restart`
6. “okay thanks, bye” — `bye`
7. “sth. more expensive plz.” — `inform`
8. “Repeat that plz.” — `repeat`
9. “Which direction is it in the city?” — `request`
10. “Repeat what you said please.” — `repeat`
11. “Chinese food.” — `inform`
12. “Hangzhoucai please.” — `inform`
13. “I love Sushi. Recommend.” — `inform`
14. “I wanna try something new.” — `reqalts`
15. “Repeat the phone number” — `repeat`
16. “Is there a double 0 in the phone number?” — `confirm`

C.2. Difficult test set 2

1. “I do not want to try this one” — `deny`
2. “Thanx, and tell me the exact location please” — `request`
3. “Maybe could you tell me something else? I do not like this one” — `reqalts`
4. “I need this restaurant address and recommend me another restaurant” — `reqalts`
5. “Recommend me three restaurants which are all with good food quality” — `reqalts`
6. “What time I can go to the restaurant that is not crowded” — `request`
7. “We need to stay from 17 to 20 o’clock, is it possible?” — `confirm`
8. “BTW, is the restaurant not crowded at 18?” — `confirm`
9. “We can to go the restaurananant you recommend. Thank you.” — `thankyou`

10. "That will be fine. Thank you so much." — **thankyou**
11. "Would you please tell me the phone number of these two restaurant?" — **request**
12. "Just please give me three names that are Italian ones." — **reqalts**
13. "One of my friends said he does not like Indian food, please let me try Thai food."
— **inform**
14. "We will go to the park in the south, would you please recommend me a restaurant
in the south?" — **inform**
15. "Okay, we will try the first one." — **affirm**
16. "Not south, north please." — **inform**

Appendix D. Contributions

Member	Main responsibility	Time/reflection
Ruben Wijmenga	Loading data and code structuring	5 hours
	Rule-based classifier	3 hours
	Logistic regression classifier	3 hours
	Final report pass	4 hours
Anouk van Ladesteijn	MLP classifier	2 hours
	Code cleaning and formatting	8 hours
	Interactive prompt and README	3 hours
	Report introduction	2 hours
Huub Statius Muller	Frozen-embedding classifiers	12 hours
	Report experiments and results	2 hours
Yanpeng Wang	Model evaluation	4 hours
	Report system architecture	5 hours
	Report appendices	4 hours

Table 2: Current division of work. Each member should add time spent and a short reflection before final submission.

Appendix E. AI disclosure statement

We, Anouk van Ladesteijn, Huub Statius Muller, Yanpeng Wang, and Ruben Wijmenga, confirm that the submitted work for the course *Methods in AI research* is our own work. We used AI tools to assist with the submitted work as described below, checked the generated output, and accept responsibility for its accuracy and validity.

Names and signatures of all students: _____

Date: September 24, 2026

Tool	Purpose	Used by
OpenAI Codex	Code review and optimization, evaluation/result-persistence assistance, and report drafting	Yanpeng Wang
ChatGPT	Asked for recommended libraries and advise for how to optimise the frozen embeddings as they were slow (not actual code). Also created the appendix for difficult test cases	Huub
Claude	Suggestions for cleaning the code and checking the code and README for errors and inconsistencies	Anouk

Table 3: Overview of AI tools used